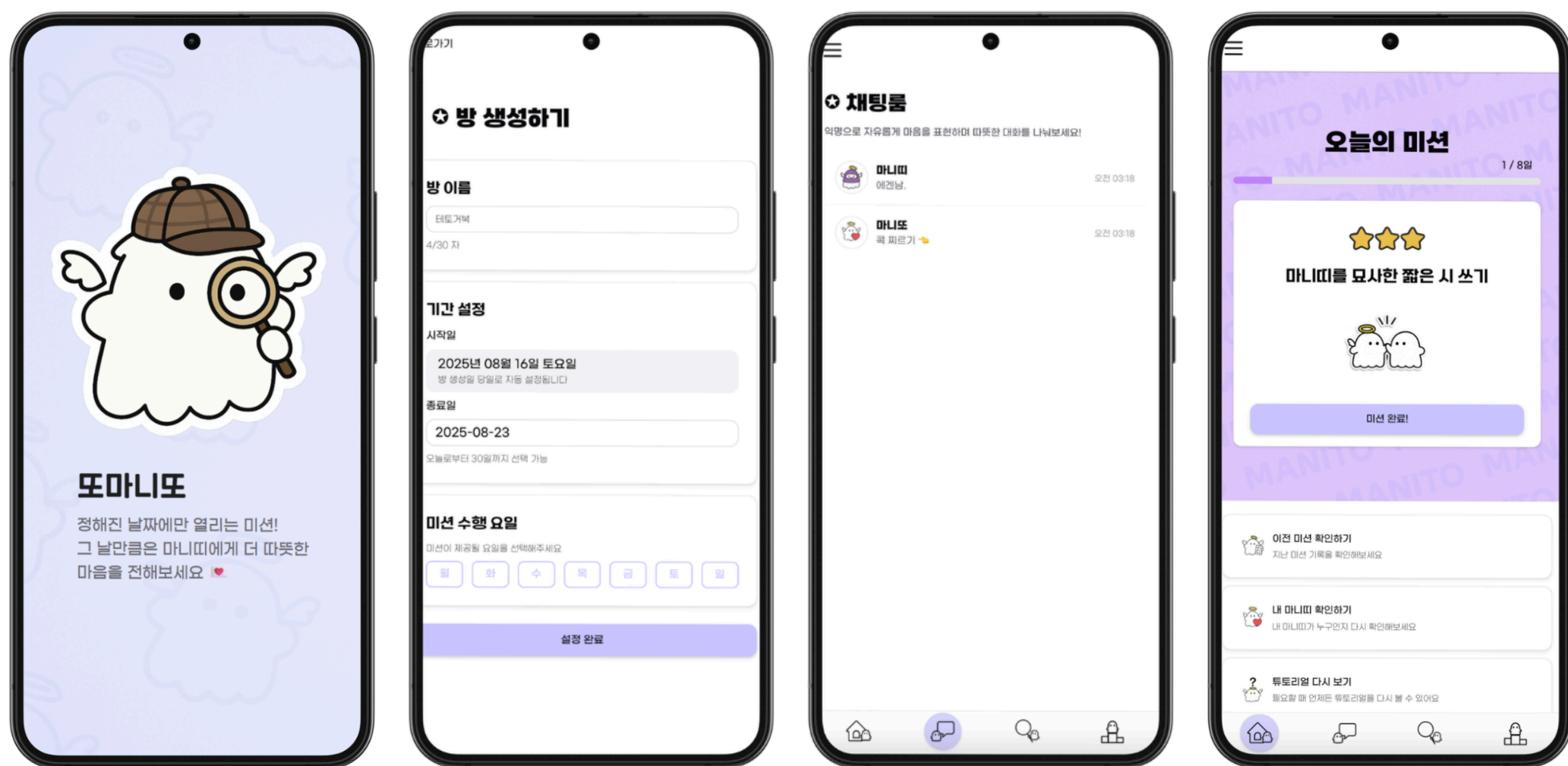




서비스

실시간 익명 채팅 마니또 게임 모바일 웹

익명 1:1 채팅·미션·푸시 알림·AI 성향 분석으로 사용자 간 자연스러운 교류를 지원하는 소셜 웹앱

[GitHub Repository](#)

작업 기간

2025. 07. 14 - 2025. 08. 15 (5주)
SSAFY 13기 공동 프로젝트 (웹기술)

기술 스택



구성원

총 6명
FE 3명 · BE 2명 (본인 포함) · Infra 1명

역할 — Backend

실시간 1:1 채팅

WebSocket+STOMP 라우팅 / MongoDB 채팅 메시지 영속

JWT Principal 통일

WebSocket 인증 충돌 → JWT 기반 Principal 주입으로 1:1 라우팅 안정화

인증/보안

JWT + OAuth2 / Redis RTR / Nginx·SSL·CORS 환경 대응

AI 성향 분석

GPT-4.1 mini 연동 — 채팅 참여자 성향 분석 API 제공

PWA 푸시

Firebase FCM — 모바일 웹 실시간 알림 + PWA 설치

프로젝트 회고

- **WebSocket 인증 충돌 → JWT Principal 주입으로 재설계**
 - STOMP 핸드셰이크 + 채널 메시지에서 동일한 사용자 식별 보장
 - Principal.getName() = userId 통일로 1:1 라우팅 안정화
- **3DB 분리 — MySQL(메타) · MongoDB(메시지) · Redis(세션·캐시)**
 - 채팅 영속과 메타 데이터의 갱신 빈도·정렬 비용 차이를 저장소로 분리
- **운영 환경 보안 이슈 해결 — Nginx·SSL·CORS 실서비스 경험**
 - JWT 쿠키 처리 + Refresh Token 회전 + 세션 안정화
- **모바일 웹 PWA 적용으로 앱 체감 향상 + 푸시 알림 자동 수신**

PROBLEM → SOLUTION

마니또 ↔ 마니띠 메시지가 두 사람 사이에서만 오가야 게임이 성립한다. HTTP 인증(`SecurityContext` · `ThreadLocal`)과 WebSocket 채널 (비동기)이 분리되어 `Principal.getName()` 이 비어 1:1 라우팅 실패 → **JwtUserDetails 단일 객체로 통일**해 해결.

Before vs After

BEFORE

HTTP-only Principal — WebSocket 채널에서 미주입

[HTTP 요청]
└─ JwtFilter → SecurityContext.set(auth) ✓
[WebSocket 메시지]
└─ Principal == null ✕
└─ /user/queue/chat.5 라우팅 실패

- ✕ HTTP의 ThreadLocal 인증 정보가 비동기 WS 채널에 전파되지 않음
- ✕ Principal.getName() = null → convertAndSendToUser 무동작
- ✕ 다른 방으로 메시지가 새거나 도착 안 함 → 게임성 붕괴

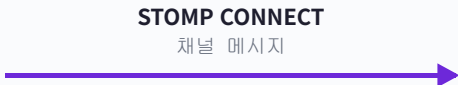
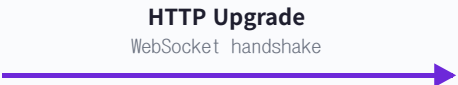
AFTER

JwtUserDetails — UserDetails + Principal 동시 구현

[Handshake] JwtHandshakeInterceptor
└─ JWT 검증 → JwtUserDetails 생성
└─ attributes.put("user", principal)
[STOMP CONNECT] WsAuthChannelInterceptor
└─ SimpUserHeader = JwtUserDetails
└─ Principal.getName() = userId ✓

- ✓ 2단계 인터셉터 — Handshake(HTTP 단계) + ChannelInterceptor(STOMP 단계)
- ✓ 커스텀 Principal — JwtUserDetails가 동시에 인증 객체+사용자 식별자
- ✓ Principal.getName() = userId 통일 → 1:1 라우팅 안정화

2단계 인터셉터 — Handshake + Channel



```
global/security/JwtUserDetails.java
UserDetails + Principal 동시 구현

@Getter @RequiredArgsConstructor
public class JwtUserDetails
    implements UserDetails, Principal {

    private final Long userId;
    private final String nickname;
    private final Collection<? extends GrantedAuthority> authorities;

    // ★ Principal.getName() == userId - 1:1 라우팅 키
    @Override
    public String getName() {
        return String.valueOf(userId);
    }

    // UserDetails 메서드들 (생략)
    @Override public String getUsername() { return getName(); }
    @Override public String getPassword() { return null; }
}

// JwtHandshakeInterceptor - HTTP 업그레이드 단계
public boolean beforeHandshake(ServerHttpRequest req, ...,
                                Map<String, Object> attributes) {
    String token = extractToken(req);
    JwtUserDetails principal = jwtProvider.parse(token);
    attributes.put("user", principal); // ★ STOMP 단계로 전달
    return true;
}

// WsAuthChannelInterceptor - STOMP CONNECT 단계
@Override
public Message<?> preSend(Message<?> msg, MessageChannel ch) {
    StompHeaderAccessor acc = StompHeaderAccessor.wrap(msg);
    if (StompCommand.CONNECT.equals(acc.getCommand())) {
        JwtUserDetails p = (JwtUserDetails) acc.getSessionAttributes().get("user");
        acc.setUser(p); // ★ Principal.getName() = userId 일관 주입
    }
    return msg;
}
```

1:1 라우팅

메시지 정확도
/user/queue/chat 채널에 정확한 사용자만 도달

단일 객체

JwtUserDetails 통합
UserDetails + Principal 동시 구현으로 HTTP·WS 컨텍스트 통일

2 단계

인터셉터 설계
Handshake (HTTP) + Channel (STOMP) 책임 분리

DESIGN DECISION

"채팅"이라는 단어로 묶이지만 **메타 데이터** · **메시지** · **세션**은 갱신 빈도와 일관성 요구가 다르다. 단일 DB로 끼워 맞추는 대신 **3DB 책임 분리**로 확장성과 성능을 동시 확보.

3DB 분리 — 책임에 따른 저장소 선택

RDB MySQL 메타 데이터 · 강한 일관성 member · room · chatting_room · manito_assignment · mission · fcm_token 외래 키 + 트랜잭션이 필요한 도메인. 매칭 · 인증 · 미션 영속.	NoSQL MongoDB 채팅 메시지 · append-only chat_message { senderUserId, chattingRoomId, type, content, sentAt } 빠른 append + 시간 역순 페이지네이션. 외래 키 없이 문서 단위로 자유롭게 저장.	In-Memory Redis 세션 · 캐시 · 토큰 refresh:{userId} blacklist:{jti} fcm:cache:{userId} 빠른 read/write + TTL. RTR(Refresh Token Rotation) 토큰 회전, 인증 세션 안정화.
---	--	--

이벤트별 DB 라우팅 — 같은 "채팅"이라도 다른 저장소

로그인 JWT 발급	member 조회	—	refresh 저장 (TTL)
방 생성 매칭	room + chatting_room INSERT	—	—
채팅 발송 STOMP	lastMessage 갱신	chat_message append	—
채팅 페이지 역순 조회	—	chat_message find (sentAt DESC)	—
로그아웃 토큰 무효	—	—	blacklist 등록 + refresh 삭제

A AI 성향 분석 (GPT-4.1 mini)

[입력] 마니띠 채팅 메시지 30건
[프롬프트] "이 사용자의 성향 Top 3"
[모델] gpt-4.1-mini (OpenAI)
↓
[출력] { friendly, curious, careful }
↓ MongoDB analysis 컬렉션 저장

- 비식별화 — 메시지 내용만 추출, userId 제외하여 프롬프트 구성
- 호출 제한 — 일일 1회 + 캐시로 토큰 비용 통제
- 마니띠 본인에게만 분석 결과 노출 (권한 검증)

B Firebase FCM PWA 푸시

[웹앱] Service Worker 등록 + 토큰 발급
↓ MySQL fcm_token 저장
[이벤트] 새 메시지 / 미션 도착
↓ FCM API 호출
[디바이스] 백그라운드 푸시 알림
↓ 앱 미설치 상태에서도 수신

- PWA 설치 — 모바일 웹을 앱처럼 홈 화면 추가
- 오프라인 대응 — Service Worker 캐싱으로 빠른 재진입
- SSL 필수 — 운영 Nginx + Let's Encrypt 인증서 적용